Skib Language Specification

ISO Skib DRAFT 1987

Current Status: Kanban Draft / WIP

Contents

1. Introduction 2
   1.1. Skib and SkibCore 2
   1.2. Core Principles 2
2. Type System 2
   2.1. Core Constructs 2
   2.2. Type Inference 2
   2.3. Typeclasses (Traits) 2
3. Functions and Syntax 3
   3.1. Application and Currying 3
   3.2. Pipeline Operator 3
   3.3. Tail Call Optimization (TCO) 3
4. Memory Management 3
   4.1. Allocators and `defer` 3
   4.2. Smart Pointers 4
   4.3. Implicit Parameters (Contexts) 4
5. Metaprogramming 4
   5.1. Procedural Macros 4

1. Introduction

Skib is a systems functional programming language. The core ideology of Skib is predictability, simple rules, and the absence of implicit behavior (acting as an anti-C++ approach). The language provides high-level abstractions (Algebraic Data Types, Pattern Matching, Typeclasses) while maintaining total manual control over memory and hardware resources (inspired by Zig).

1.1. Skib and SkibCore

The compilation pipeline is conceptually divided into two tiers:

1. **Skib:** The full-featured language containing implicits, procedural macros, and syntactic sugar.
2. **SkibCore:** A minimalist intermediate representation (IR). All complex Skib features are desugared into SkibCore, which operates strictly on ADTs, structs, functions, and explicit memory allocations. SkibCore is then compiled into ANSI C98 or LLVM IR.

1.2. Core Principles

- **No Garbage Collector (GC):** All memory is managed manually.
- **No Borrow Checker:** The compiler does not track reference lifetimes (unlike Rust). Avoiding memory leaks and dangling pointers is solely the programmer's responsibility.
- **Explicit over Implicit:** The language rejects features like implicit function currying and silent C++-style constructor overloads, which obscure performance costs and trigger hidden heap allocations.

2. Type System

The Skib type system is primarily structural and nominal, heavily featuring Algebraic Data Types (ADTs) and structs, with ad-hoc polymorphism achieved through typeclasses.

2.1. Core Constructs

- **Structs:** Standard product types used for data grouping.
- **Algebraic Data Types (ADTs):** Sum types with full Pattern Matching support.

2.2. Type Inference

Skib implements **local type inference**.

- Global type inference (e.g., global Hindley-Milner) is strictly **rejected** because it leads to confusing error messages in unrelated parts of the codebase.
- Function signatures (including generics) must be explicitly annotated.
- Inside a function body, the types of variable bindings (`let`) are inferred top-down.

2.3. Typeclasses (Traits)

Typeclasses are used for ad-hoc polymorphism. They allow defining shared behavior for types (e.g., custom allocators, resource release via `deinit`, or smart pointer logic inside the standard library).

TODO: Unresolved Type System Issues

1. **Higher-Kinded Types (HKT):** Should the language support HKT (`* -> *`) for typeclasses to represent concepts like Monads? (Debated).
2. **Recursive ADTs:** Since memory is manual and types are unboxed by default, how are recursive ADTs constructed? Do they require an explicit `Ptr<T>` wrapper inside the variant definition?
3. **Subtyping:** Structural subtyping was mentioned but currently considered too complex for the systems programming paradigm.

3. Functions and Syntax

3.1. Application and Currying

Function application uses explicit parentheses: `f(a, b, c)`.

- **No Space Application:** Haskell-style space application (`f a b c`) is rejected.
- **No Auto-Currying:** Automatic currying creates implicit closures and causes severe performance drops due to hidden heap allocations.
- **Custom Operators:** The language supports custom user-defined operators (which act as standard functions).

3.2. Pipeline Operator

To prevent deep nesting of parentheses `(())))` when calling multiple functions, Skib provides a pipeline operator `|>`.

```
skolemization(x)
  |> skibidization(4)
  |> kekszation(5)
```

3.3. Tail Call Optimization (TCO)

Tail recursion must be optimized into a loop to prevent stack overflows and save stack frame allocations. A specific keyword or annotation (e.g., `tailcall`) will be required so the compiler can throw an error if the optimization is mathematically impossible in a given context.

TODO: Unresolved Function Issues

1. **Closures and Environment Capture:** If auto-currying is dead, what is the exact syntax for closures `\x -> x + y`?
2. **Closure Allocation:** If a closure captures a local variable, does it live on the stack or the heap? How is the allocator passed to it without breaking the "explicit memory" rule?

4. Memory Management

Skib utilizes a purely manual memory management model. There is no affine type system, no move semantics by default, and no borrow checker.

4.1. Allocators and `defer`

Memory is controlled using Zig-style allocators. Standard `malloc` is considered too slow and error-prone; instead, users pass explicit `Allocator` or `Arena` instances. To clean up resources at the end of a scope, Skib uses the `defer` keyword.

```
let arena = Arena.init();
defer arena.deinit();
```

4.2. Smart Pointers

The standard library will provide explicit smart pointers (e.g., similar to `Box` or `Unique`). Since there is no borrow checker, ownership logic is maintained by explicit traits (typeclasses) handling `deinit` when the pointer is explicitly dropped or goes out of scope.

4.3. Implicit Parameters (Contexts)

To avoid threading an `Allocator` through every single function call manually, Skib allows strictly scoped implicit parameters.

- Implicits are resolved via the calling function's signature.
- Searching the global scope or class scope for implicits (Scala-style) is **forbidden**.
- Implicits are declared at the end of a function call using a specific keyword (e.g., `ex`), preventing ambiguity with regular arguments.

```
// Overriding the implicit allocator explicitly at the call site
skolemization(x, ex alloc = &myAlloc)
```

TODO: Unresolved Memory Issues

1. **Move Semantics:** Can a smart pointer be safely returned from a function without triggering C++-style rule-of-3 overhead?
2. **Duplicate Implicit Types:** If a function requires two different allocators (e.g., `scratch: &Allocator`, `perm: &Allocator`), how does the implicit resolution distinguish between them without relying purely on parameter names?

5. Metaprogramming

Skib requires a powerful metaprogramming system primarily to handle automatic code generation for ADTs and Structs (e.g., deriving `Serialize`, `Eq`, `Show`).

5.1. Procedural Macros

Macros operate as `Tokens -> Tokens` transformations (similar to Rust). They are executed lexically before type inference.

```
@derive(Serialize)
struct S {
  @serde(skip)
  hidden: Int
}
```

TODO: Unresolved Metaprogramming Issues

1. **Tokens vs. Type Information:** A raw `Tokens -> Tokens` macro cannot query type information (since inference hasn't happened yet). Should Skib use Zig-style `comptime` reflection instead to loop through struct fields dynamically, or stick to raw token parsing?
2. **Macro Cycles:** If a macro requires type info to expand, and the type inference requires the macro to expand first, how does the compiler break the cyclic dependency?